

Programming for Deep Learning (CO3027)

Final Project Proposal

Robust CAPTCHA Recognition under Corrupted Conditions: Sequence-based Recognition vs Position-wise Character Classification

Course: Programming for Deep Learning (CO3027)

Team: 吳睿嫻 / 葉俊廷 / 盛正璿

1. Topic

Robust CAPTCHA Recognition under Corrupted Conditions: Sequence-based Recognition vs Position-wise Character Classification

2. Brief Introduction of This Project

Text-based CAPTCHA is a common human verification mechanism, but its visual corruption patterns such as noise, blur, interference lines, rotation, occlusion, overlapping characters, and complex backgrounds make recognition difficult. This project studies CAPTCHA recognition in a controlled experimental setting and focuses on how different recognition strategies behave under clean, corrupted, and restored inputs.

The project compares two recognition paradigms and one restoration module:

- Image restoration and data preparation
- Sequence-based CAPTCHA recognition
- Position-wise multi-head character classification

Research questions:

- RQ1: Can image restoration improve recognition accuracy under corrupted conditions?
- RQ2: Which recognition strategy is more robust: CRNN-based sequence recognition or CNN multi-head character classification?
- RQ3: Can contrastive and self-supervised feature learning improve robustness under corruption?

3. Related Work

3.1 Image Restoration for Corrupted CAPTCHA

Image restoration is an important component of this project because corrupted CAPTCHA images often contain noise, blur, rotation, interference lines, and partial occlusion. For the first restoration branch, we refer to the Denoising Autoencoder (DAE) of Vincent et al. [1], which learns to reconstruct clean inputs from corrupted observations. This idea is directly relevant to our A-1 module, where corrupted CAPTCHA images are mapped back to cleaner versions through an encoder-decoder architecture.

For the second restoration branch, we adopt DnCNN [2], a residual-learning-based denoising model that predicts noise residuals instead of directly reconstructing the clean image. This design is closely related to our A-2 module, where restored CAPTCHA images are obtained by subtracting predicted residual noise from corrupted inputs.

Because our restoration experiments also evaluate structural image similarity, we further rely on SSIM [3] as an important metric for comparing clean, corrupted, and restored CAPTCHA images.

3.2 Sequence-based CAPTCHA Recognition

For sequence-based recognition, our work is mainly based on the CRNN framework proposed by Shi, Bai, and Yao [5]. CRNN combines convolutional feature extraction with recurrent sequence modeling, making it suitable for unsegmented text recognition and overlapping characters. This is directly aligned with our B-1 module, where the whole CAPTCHA image is recognized as a sequence without explicit character segmentation.

To train sequence models without character-level alignment, we adopt Connectionist Temporal Classification (CTC) introduced by Graves et al. [4]. CTC is particularly suitable for our task because the model only needs the full CAPTCHA string label rather than the exact position of each character.

In addition to CRNN, our B-2 model incorporates a Transformer encoder for stronger global feature learning and long-range dependency modeling. For this part, we refer to the Vision Transformer work of Dosovitskiy et al. [6], which shows the effectiveness of transformer-based visual representations.

3.3 Position-wise Multi-head Character Classification

For position-wise CAPTCHA recognition, our method follows a fixed-length multi-head classification design, where each output head predicts one character position independently. To strengthen the backbone in the improved version of our model, we refer to ResNet [7], which provides a strong and widely used convolutional feature extractor. This is especially relevant to our C-2 module, where a better CNN backbone is used for more robust feature learning.

Since CAPTCHA images may contain rotation, misalignment, and geometric distortion, we also refer to Spatial Transformer Networks [8]. STN is relevant because it provides a way to improve invariance to spatial transformation before character classification. This is useful for the improved position-wise model under corrupted conditions.

3.4 Self-supervised and Semi-supervised Learning

To improve robustness under corruption, all three modules make use of self-supervised or contrastive learning ideas. Our main reference is SimCLR [9], which learns invariant feature representations by bringing augmented views of the same sample closer while pushing different samples apart. This directly supports our use of positive pairs, negative pairs, and augmentation consistency learning in A, B, and C.

3.5 Optimization and Training Strategy

For optimization, we use Adam [10], because it is a standard and effective optimizer for deep learning training.

For learning-rate tuning, we refer to Cyclical Learning Rates [11], which highlights the importance of learning-rate search and scheduling in model performance. This is relevant to our experiments because we

compare multiple learning-rate settings and analyze how learning-rate adjustment affects convergence and robustness.

4. Dataset Specification

The CAPTCHA dataset is generated with the Python package ``captcha.image.ImageCaptcha``. Each sample contains a fixed-length 5-character string composed of uppercase letters A-Z and digits 0-9.

Item	Setting
Image size	160 x 60
CAPTCHA length	5
Character set	A-Z and 0-9
Random seed	3027
Dataset size	100,000 clean CAPTCHA images
Split	70% train / 15% validation / 15% test
Filename format	index_text.png, e.g. 1023_A7K9P.png

Data preparation stages:

1. Generate clean CAPTCHA images.
2. Create corrupted sets with Gaussian noise, motion blur, Gaussian blur, rotation, interference lines, background clutter, and partial occlusion.
3. Apply restoration models to create restored CAPTCHA images for downstream recognition experiments.

5. Team Project Plan / Methodology / Coding Architecture

The project is organized into three modules. The proposal below updates each module with the concrete technical scope described in A.docx, B.docx, and C.docx.

5.1 Module A - Image Restoration and Data Preparation

This module is handled by 吳睿嫻. Its purpose is to generate corrupted CAPTCHA images, restore them with deep learning models, and provide clean/corrupted/restored datasets to Modules B and C for recognition experiments.

Two restoration models will be implemented:

- Denoising Autoencoder (DAE): encoder-decoder reconstruction from corrupted CAPTCHA to clean CAPTCHA.
- DnCNN: residual-learning denoising model that predicts noise residuals and recovers restored CAPTCHA images.

Corruption generation strategy:

- Gaussian noise with multiple sigma levels
- Blur with multiple kernel sizes
- Rotation with multiple severity levels
- Interference lines with multiple counts

- Partial occlusion with multiple ratios

A will study multiple learning settings for restoration:

- Supervised learning on corrupted -> clean image pairs
- Unsupervised learning such as Noise2Noise or corruption-distribution learning
- Semi-supervised learning with limited clean pairs and consistency regularization
- Self-supervised learning such as contrastive learning or masked pixel prediction

Key questions for Module A:

- Which restoration model produces the best restored CAPTCHA quality?
- Can restoration improve downstream recognition accuracy?
- Which corruption type is the hardest to repair?

Metric	Description
Reconstruction Loss	Training loss for restoration quality
PSNR	Peak Signal-to-Noise Ratio
SSIM	Structural Similarity Index
Visual Comparison	Side-by-side clean / corrupted / restored examples

5.2 Module B - Sequence-based CAPTCHA Recognition

This module is handled by 葉俊廷. It recognizes the complete CAPTCHA string without manually segmenting characters, making it suitable for overlapping characters and irregular spacing.

Module B includes two sequence models:

- B-1 CRNN with CTC Loss: CNN feature extractor -> sequence feature map -> RNN/LSTM/GRU -> CTC -> predicted CAPTCHA string.
- B-2 CNN + Transformer Encoder + CTC: sequence modeling with stronger long-range dependency learning.

B will explore four learning settings:

- Supervised learning with image + full sequence label and CTC Loss
- Unsupervised contrastive learning to pretrain the CNN encoder with positive/negative CAPTCHA pairs
- Semi-supervised learning with labeled CAPTCHA and pseudo-labels on unlabeled data
- Self-supervised augmentation consistency learning under blur, noise, rotation, and brightness changes

B focuses on:

- Sequence recognition accuracy
- Robustness under corrupted CAPTCHA conditions
- Recognition performance on restored CAPTCHA images

Model	Train Data	Test Data
Sequence CNN	Clean	Clean
Sequence CNN	Clean	Corrupted
Sequence CNN	Restored	Corrupted

Model	Train Data	Test Data
Sequence CNN	Restored	Restored
Transformer OCR CNN	Clean	Clean
Transformer OCR CNN	Clean	Corrupted
Transformer OCR CNN	Restored	Corrupted
Transformer OCR CNN	Restored	Restored

5.3 Module C - Position-wise Multi-head CNN

This module is handled by 盛正璿. It treats CAPTCHA as a fixed-length string and predicts each character position independently with a shared encoder and multiple heads.

Module C includes two position-wise variants:

- C-1 Pure Multi-head CNN: shared CNN encoder + 5 classification heads, each head performing 36-class prediction.
- C-2 Attention + Multi-head CNN: adds attention to focus on character regions and reduce background interference.

Advantages of the multi-head design:

- Simple and stable training
- Fast inference
- Natural per-position analysis
- Suitable for fixed-length CAPTCHA

Limitations to analyze:

- Sensitive to variable-length strings
- More affected by overlapping characters or severe position shifts

C will explore four learning settings:

- Supervised learning with five position labels and summed cross-entropy loss
- Unsupervised contrastive learning for robust shared feature representation
- Semi-supervised learning with pseudo-labels for unlabeled CAPTCHA
- Self-supervised augmentation consistency learning for position-wise feature stability

Additional engineering requirements for Module C:

- Track per-position accuracy, character accuracy, sequence accuracy, edit distance, robustness drop, and restoration gain
- Keep full experiment artifacts, including checkpoints, metrics, plots, and failure cases
- Record learning-rate changes and decision reasons during training

Metric	Description
Per-position Accuracy	Accuracy of each of the five character positions
Character Accuracy	Average accuracy across all characters
Sequence Accuracy	Whether the whole CAPTCHA is exactly correct
Edit Distance	Difference between predicted string and ground

Metric	Description
	truth
Robustness Drop	Accuracy decrease after corruption
Restoration Gain	Accuracy increase after restoration

6. Evaluation Metrics

The final evaluation combines restoration quality, recognition accuracy, and robustness analysis.

6.1 Restoration Metrics

Metric	Description
Reconstruction Loss	Restoration training loss
PSNR	Pixel-level restoration quality
SSIM	Structural similarity
Visual Comparison	Clean / corrupted / restored examples

6.2 Recognition Metrics

Metric	Description
Character Accuracy	Whether each predicted character is correct
Sequence Accuracy	Whether the full CAPTCHA string is exactly correct
Edit Distance	Distance between prediction and ground truth
Per-position Accuracy	Accuracy of positions 1 to 5 for Module C

6.3 Robustness Metrics

Metric	Description
Clean Accuracy	Accuracy on the clean test set
Noise Accuracy	Accuracy on noisy CAPTCHA images
Blur Accuracy	Accuracy on blurred CAPTCHA images
Rotation Accuracy	Accuracy on rotated CAPTCHA images
Interference-line Accuracy	Accuracy on line-corrupted CAPTCHA images
Restored Accuracy	Accuracy after restoration
Robustness Drop	Clean Accuracy - Corrupted Accuracy
Restoration Gain	Restored Accuracy - Corrupted Accuracy

7. Work Assignment and Five-Week Schedule

Member	Main Responsibility	AI / ML / DL Method
吳睿嫻	CAPTCHA corruption generation and image restoration	Denoising Autoencoder, DnCNN, restoration-oriented supervised / semi-supervised / self-supervised learning
葉俊廷	Sequence-based CAPTCHA recognition	CRNN, Transformer Encoder, CTC Loss, Contrastive Learning, Semi-supervised and Self-

Member	Main Responsibility	AI / ML / DL Method
		supervised learning
盛正璿	Position-wise character classification and robustness learning	Pure Multi-head CNN, Attention + Multi-head CNN, Contrastive Learning, Pseudo-labeling, Augmentation consistency learning

Week	Focus	Expected Output
Week 1	Proposal discussion and project planning	Finalize proposal, research questions, dataset plan, and division of responsibilities.
Week 2	Dataset generation and baseline setup	Generate clean/corrupted CAPTCHA data, build CRNN baseline, and build multi-head CNN baseline.
Week 3	Main model implementation	Implement DAE and DnCNN, train sequence models, train multi-head CNN models, and test feature pretraining.
Week 4	Robustness evaluation and comparison	Analyze restoration quality, evaluate sequence recognition, evaluate position-wise robustness, and compare methods.
Week 5	Final integration and report	Integrate results, prepare report/slides, finalize figures, code, and comparison tables.

8. Final Deliverables

- A clean CAPTCHA generator together with corrupted and restored evaluation sets
- DAE and DnCNN restoration models with restoration-quality analysis
- CRNN and Transformer-based sequence-recognition models
- Pure Multi-head CNN and Attention + Multi-head CNN classifiers
- Evaluation tables under clean, corrupted, and restored conditions
- Robustness Drop and Restoration Gain analysis
- Stored checkpoints, metrics, plots, and failure-analysis records for reproducible experiments
- Final report and presentation slides

9. References

- [1] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in Proc. ICML, 2008.
- [2] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," IEEE Trans. Image Process., vol. 26, no. 7, pp. 3142-3155, 2017.

- [3] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600-612, 2004.
- [4] A. Graves, S. Fernandez, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. ICML*, 2006.
- [5] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298-2304, 2017.
- [6] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016.
- [8] M. Jaderberg, K. Simonyan, A. Zisserman, and K. Kavukcuoglu, "Spatial transformer networks," in *Proc. NeurIPS*, 2015.
- [9] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. ICML*, 2020.
- [10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
- [11] L. N. Smith, "Cyclical learning rates for training neural networks," in *Proc. WACV*, 2017.